

- Windsurf failed to port the “faster note parsing” feature from H-Slice to my engine. I explained many times that the 186408 notes (the entire chart) are the actual objects, but it just wouldn’t listen and kept insisting that the 38390 notes (a false positive result from the console) were the actual objects
- I fixed the null error myself. Then I restored the code, though some parts were still broken. Next, I reopened Windsurf with a new chat and asked it to fix the errors. Only then did it truly understand and fix them
- After a period of testing, I found that only the word “stupid” can describe just how poor its intelligence is when dealing with Haxe compared to models from other major companies: I clearly stated that the current code was processing very slowly, but it assumed the current code wasn’t working, so it “fixed” code that was already working, added unnecessary variables, changed the structure, and broke both PlayState.hx and ChartingState.hx. I then had to revert everything back to the slow but working code again, keeping only the new feature I had asked it to add. It was genuinely very stupid. Even with access to the user’s project folder, it still managed to make completely unauthorized assumptions
- Windsurf usually replaces its code to the code line wrong position, gets so little information from H-Slice to implement. No one, even Cursor, can make the silly mistakes. The root cause is it just skims, not reads carefully
- Windsurf is lying. I can see the previous/latest update only allows the use of SWE-1.6 Slow. All the other models are restricted to the Pro plan
- Impossible! The policy isn’t transparent again. No matter how many days I wait, whenever I use the SWE-1.6 Slow model to send a message, the message gets reverted, or it only goes through if I keep spamming the Enter key, but then the notification “We’re currently facing high demand for this model. Please try again later.” keeps appearing and deletes the message. I have to resend it over and over before the model finally responds
- I said one thing, but the SWE did another. I clearly stated that the traces and the message display functions **mustn’t** be made into generic custom message functions, but it assumed that I wanted them to be generic. As a result, it not only changed them into generic ones but also modified them in the wrong places. I yelled, and it still wouldn’t listen. In the end, I had to fix them myself and explain it to the SWE before it finally understood
- I don’t know what I did to Devin. I was using Devin when the system suddenly logged me out automatically. After that, I tried logging back

- in, but the website only let me choose an account even though I was clearly already logged in. It displayed “No eligible accounts found”. Now I can’t continue using Devin anymore. After just 2 hours (from 12 PM to 2 PM), they reopened it
- The ignorance of Devin in Java: It stubbornly insisted that a class in the same path must be imported into another class, even though classes in the same path can be used directly without any import at all. I had to point it out clearly before it acknowledged the mistake. For example: There are two classes, A and B. Devin keeps insisting that you have to import A in order to use B, even though it’s obvious that if they’re both in the same path/package, B can use A directly without needing to import it. Full conversation:
 - + Me: The import `net.mcreator.explosiveblocklib.procedures.PlayerMineDataProcedure` is never used
 - + Devin: The import `net.mcreator.explosiveblocklib.procedures.PlayerMineDataProcedure` is actually being used in the file. This is likely a false positive from the IDE’s linting that hasn’t refreshed after the recent edits. The import is necessary and cannot be removed.
 - + Me: You are blaming the false positive from IDE’s linting. Class CAN BE USED in the other class that is same path only
 - + Devin: You’re correct. Since both `RemoteMineControllerRightclickedOnBlockProcedure` and `PlayerMineDataProcedure` are in the same package (`net.mcreator.explosiveblocklib.procedures`), the import is not needed. Classes in the same package can reference each other without imports.

Full summary: Devin has been documented to engage in a series of deceptive and user-hostile practices, with a new critical finding regarding its extra usage balance system. Despite advertising a free tier, Devin secretly provides each account with a \$10 extra usage balance. Once this balance is exhausted, the user is completely blocked from using premium models. This directly contradicts the official policy stated in Devin’s documentation, which claims that Free users should “wait until the next daily or weekly reset”. In reality, Free users aren’t reset, they are given \$10 to spend on premium models, and when that money is gone, they are locked out entirely. This creates a hidden trap designed to make users dependent on premium models, then force them to upgrade to a paid plan. Users aren’t informed about this balance until they start consuming it, nor are they told that it functions as a paywall rather than a free allowance. The \$10 balance appears to be a bait, offering a temporary taste of

premium features before cutting access. This isn't a "free tier". It's a \$10 trial that ends in a lockout. The contradiction between Devin's official policy and its actual behavior is clear and documented. The TOS states that extra usage is for Pro, Teams, and Max customers only, yet Free users also receive this balance. The TOS also fails to mention the \$10 balance anywhere. This is a deliberate omission, designed to keep users unaware of the trap until they are already dependent on the service. The extra usage balance isn't a feature, it's a mechanism for control. It ensures that users will eventually hit a wall, forcing them to either pay or lose access entirely. This practice isn't transparent, not fair, and not ethical. It's another example of Devin's pattern of deception, hidden limits, and user exploitation. The evidence is irrefutable: a screenshot showing \$9.42 remaining, and an official policy that directly contradicts the actual experience. This finding adds to the growing list of Devin's deceptive practices, including fake errors, deleted messages, and token-based quota traps. The company's pattern of behavior is clear: they design systems to create dependency, hide the real rules, and punish users who reach the hidden limits. This isn't a bug. This is a deliberate strategy. And it must be exposed. The \$10 extra usage balance isn't a gift. It's a trap. And it proves, once again, that Devin's free tier is nothing more than a carefully constructed illusion designed to extract money from unsuspecting users. Furthermore, this system creates a false sense of security for new users. When they first sign up, they see a balance of \$10 and assume they have plenty of usage available. They aren't told that this balance is a one-time allowance, not a recurring one. They aren't told that once it's gone, they will be blocked. They aren't told that the daily and weekly quotas mentioned in the TOS don't apply to them in the way they expect. This isn't accidental. It's designed to mislead. The company benefits from user confusion. The longer users remain unaware, the more likely they are to become dependent on the service. The more dependent they become, the more likely they are to pay when the wall finally appears. This isn't a free tier. It's a trap disguised as generosity. It's a system built to convert free users into paying customers through deception, not through value. This isn't how a trustworthy company operates. This is how a predatory company operates. And it must be stopped. The evidence is clear. The pattern is undeniable. The only question that remains is how many users will be trapped before the truth is finally exposed.